

Code review report v3

This is an engineering review of the whole codebase: architecture, every public Convex function, table rules, auth, locks, rate limiting, and the things earlier reports missed. Every claim names a real file and symbol so you can verify it yourself.

This version rewrites the earlier report in plain English, adds the legacy schema migration that shipped after it, and adds a new section on risks that v1 and v2 skipped over. The earlier report files in the repo root are untouched.

Contents

- 1. Architecture overview
- 2. Function inventory
- 3. Encapsulation and inheritance
- 4. Table rules
- 5. How Convex works
- 6. Validators and locks
- 7. Findings from v1, and where they stand now
- 8. Risks v1 and v2 skipped
- 9. Verification

1. Architecture overview

Stack

The app is React 19 with TanStack Start on the frontend and Convex on the backend. TypeScript throughout (tsgo, version 7). Bun is the runner. Vite with the Nitro/Vercel adapter handles SSR.

Browser	Edge / SSR	Data plane
React 19 + TanStack Start (TypeScript tsgo, Tailwind v4 tokens via app.css)	TanStack Router file routes src/routes/*.tsx (+ layout) loaders call Convex queries mutations via useMutation Convex Auth Password provider	Convex backend convex/*.ts documents + indexes queries / mutations / actions authTables (sessions, accounts)

File routes. Every file under `src/routes` is a route. `src/routes/service/job.$id.tsx` is the job detail page. `src/routes/admin/audit.tsx` is the audit log. `src/routes/auth/login.tsx` is login. The router tree is generated in `src/routeTree.gen.ts` and never hand-edited. Layouts in `__root.tsx` and `_app.tsx` provide the app shell, auth gating via `useConvexAuth` and `useCurrentUser`, and the inactivity warning modal. Route loaders read `convex/users.ts:me` and `sessionFlags` to force redirects for `mustChangePassword` and `totpExpired`.

Styling. Tailwind v4 tokens live in `src/app.css` as semantic CSS variables: `ink`, `body`, `mute`, `accent`, `bg`, `surface`, `line`, with `.dark` overrides. No hard-coded hex in components.

Backend. One file per domain in `convex/`, plus shared helpers in `convex/lib/`. The full map is below.

Request lifecycle

Here is what happens when someone clicks a button that writes data, say "Generate Final Invoice":

1. **UI call.** The component calls `useGenerateInvoiceMutation()` from `src/lib/queries.ts`, which wraps `useMutation(api.invoices.generate)`. Reads work the same way: `useQuery(jobQueries.detail(jobId))` subscribes to `convex/jobs.ts:getDetail`.
2. **Convex client.** The Convex React client (set up in `src/lib/convex.ts`) sends the call over WebSocket or HTTP to the deployment, with the auth JWT from Convex Auth attached.
3. **Auth.** The first line of the handler is almost always `await requireActiveSession(ctx, [...])` or `await requireRole(ctx, [...])` or at least `await requireUser(ctx)`. These call `getAuthUserId(ctx)`, fetch the user document, and check `active !== false`, the `lastActiveTs` inactivity window, and 2FA status. On failure they throw a `ConvexError` and the client gets a typed error.
4. **Rate limit.** Right after auth: `await enforce(ctx, class)`. This reads `rateLimits.by_key_window` for the user, class, and current window. If the bucket is full it throws `ConvexError({code: 'RATE_LIMITED', retryAfterMs})`. No writes have happened yet at this point.
5. **Validation.** `Zod.Schema.parse(args)` from `src/lib/schemas/*` runs inside the handler. Convex `v.*` validators on the function args run even earlier, before the handler body starts. If either fails, the mutation throws before touching the database.
6. **Business guards.** Things like `findApprovedFinalForJob`, `assertNotLocked`, `canTransition`, plate regex, stock checks, name similarity dedup. Thrown `ConvexError`s become toasts on the client.
7. **Writes.** `ctx.db.insert`, `patch`, or `delete`. Each mutation is one ACID transaction. If any throw happens after a write, the whole thing rolls back. One documented caveat: the best-effort `rateLimitEvents` insert also rolls back, so the durable signal for a rate-limited call is the thrown error, not the event row.
8. **Audit.** `await audit(ctx, action, entity, entityId)` inserts into `auditLogs`, still inside the same transaction, so it commits with the business write.
9. **Return and reactivity.** The return value goes back to the caller. `queryClient.invalidateQueries()` refreshes TanStack Query caches. Convex subscriptions also push updated query results to every subscribed client automatically.

```
Button click → useMutation(api.x.y) → Convex client (+JWT)
  → handler: require* → enforce → Zod+v → business guard → db.* → audit → return
  → TanStack invalidation + Convex subscription push → UI re-renders
```

convex/ module map

FILE	DOMAIN	WHAT IT OWNS
<code>convex/schema.ts</code>	schema	All tables, indexes, searchIndexes (see section 4)
<code>convex/lib/auth.ts</code>	auth helper	<code>getCurrentUser</code> , <code>requireUser</code> , <code>requireRole</code> , <code>isValidRole</code>
<code>convex/lib/session.ts</code>	session/2FA	<code>INACTIVITY_MS=30m</code> , <code>WARNING_MS=25m</code> , <code>HEARTBEAT_THROTTLE_MS=50s</code> , <code>heartbeat</code> , <code>requireActiveSession</code> , <code>sessionFlags</code>
<code>convex/lib/audit.ts</code>	audit	<code>audit(ctx, action, entity, entityId)</code> appends to <code>auditLogs</code>
<code>convex/lib/rateLimit.ts</code>	throttling	<code>RATE_LIMITS</code> , <code>windowStartFor</code> , <code>isRateLimitEnabled</code> , <code>enforce</code> , <code>enforceDedup</code>
<code>convex/lib/invoiceHelpers.ts</code>	invoicing	<code>buildLineItemsForJob</code> , <code>buildLineItemsForSalesOrder</code> , <code>nextInvoiceNumber</code> , <code>assertNotLocked</code> , <code>findApprovedFinalForJob</code> , <code>findApprovedFinalForSalesOrder</code>
<code>convex/lib/totp.ts</code>	2FA crypto	<code>base32Encode/Decode</code> , <code>generateSecret</code> , <code>generateBackupCodes</code> , <code>buildOtpauthUri</code> , <code>totpVerify</code> (window ± 1 , constant-time)

FILE	DOMAIN	WHAT IT OWNS
convex/auth.ts	Convex Auth	convexAuth({providers:[Password]}), sendVerificationRequest intentionally silent (CR-01 fix)
convex/auth.config.ts	auth config	Auth provider config for the deployment
convex/http.ts	HTTP router	httpRouter(); auth.addHttpRoutes(http), the only HTTP entry point
convex/crons.ts	crons	crons.daily("rateLimit cleanup", 03:00 UTC, internal.rateLimit.cleanup)
convex/migrations.ts	data migrations	cleanupLegacyJobs, cleanupLegacyUsers, previewLegacyJobs, previewLegacyUsers (idempotent, audited)
convex/users.ts	users/admin	me, adminExists, list, heartbeat, setRole, setActive, bootstrapFirstAdmin, adminResetPassword, changePassword, clearMustChangePassword
convex/twoFactor.ts	2FA flows	status, setup, verifySetup, verifyLogin, disable, regenerateBackupCodes, adminReset
convex/customers.ts	service/customers	search, get, getWithVehicles, create, update (phone dedup + levenshtein)
convex/vehicles.ts	inventory	get, byCustomer, byPlate, inventory, create, adjustStock, update (plate uppercase + regex)
convex/vehicleBrands.ts	brands	list, create, update, remove (normalizedName uniqueness)
convex/parts.ts	parts	list, get, lowStock, search, categories, brands, movements, createPart, updatePart, adjustStock, importParts
convex/jobs.ts	jobs	getDetail, byStatus, openCount, dashboardSummary, byCustomer, checkIn, diagnose, markReady, complete, markPaid, reverseReady, addJobItem, removeJobItem, internal syncInvoiceForJob
convex/appointments.ts	appointments	list, listRange, upcoming, get, create, markCheckedIn, cancel
convex/backfillPlates.ts	migration	backfillPlates (admin-only idempotent uppercasing of plates)
convex/leads.ts	sales/leads	list, search, get, create, updateStage, logFollowUp
convex/salesOrders.ts	sales/orders	get, list, byVehicle, byLead, create (auto-reserve), complete, cancel, addPayment
convex/deliveries.ts	sales/deliveries	get, getBySalesOrder, complete (checklist)
convex/invoices.ts	invoicing	getByJob, listByJob, listBySalesOrder, getById, generate, generateSales, regenerate, approve, createEstimate, updateEstimate, approveEstimate, rejectEstimate, convertEstimateToFinal, adminUnlock
convex/payments.ts	payments	byInvoice, record (approved final only, balance cap, dedup)
convex/labourTypes.ts	labour	list, create, update, remove
convex/settings.ts	settings	get, setVatRate

FILE	DOMAIN	WHAT IT OWNS
<code>convex/rateLimit.ts</code>	throttling admin	<code>listEvents</code> , <code>getStatus</code> , <code>setEnabled</code> , <code>logEvent</code> (internal), <code>cleanup</code> (internal, cron)
<code>convex/activityLogs.ts</code>	activity	<code>log</code> , <code>list</code> (UA parsing)
<code>convex/auditLogs.ts</code>	audit	<code>list</code> , <code>distinctActions</code>
<code>convex/seed.ts</code>	demo data	<code>seedData</code> (mutation) + <code>seed</code> (action wrapper) + <code>checkAuthAccounts</code> (query)
<code>convex/seedAdvanced.ts</code>	demo data	<code>seedAdvanced</code> , <code>verify</code>

Zod schemas live in `src/lib/schemas/*` and are imported by both client forms and Convex handlers. One source of truth for field constraints.

2. Function inventory

This section lists every public `export const` in `convex/` with its auth gate, validators, rate-limit class, and the shared helpers it calls.

Shared helpers in `convex/lib/`

HELPER	FILE AND SIGNATURE	CALLED FROM
<code>getCurrentUser(ctx)</code>	<code>convex/lib/auth.ts</code>	<code>requireUser</code> , <code>requireRole</code> , <code>requireActiveSession</code> , <code>sessionFlags</code> , <code>audit</code> , <code>rateLimit.enforce</code> , <code>users.me</code> , seed helpers
<code>requireUser(ctx)</code>	<code>convex/lib/auth.ts</code>	Every query and every mutation (indirectly via <code>requireRole</code> / <code>requireActiveSession</code>); explicit in <code>invoices.getById</code> , <code>parts.adjustStock</code> , <code>users.changePassword</code>
<code>requireRole(ctx, roles)</code>	<code>convex/lib/auth.ts</code>	Every admin/privileged mutation, <code>auditLogs.list</code> , <code>activityLogs.list</code> , <code>rateLimit.listEvents/setEnabled</code>
<code>requireActiveSession(ctx, roles)</code>	<code>convex/lib/session.ts</code>	All write mutations that must respect timeout and 2FA: <code>customers.create/update</code> , <code>vehicles.create/adjustStock/update</code> , <code>vehicleBrands.*</code> , <code>appointments.*</code> , <code>parts.*</code> , <code>jobs.checkIn/diagnose/markReady/complete/markPaid/reverseReady/addJobItem/removeJobItem</code> , <code>invoices.generate/regenerate/approve</code> , <code>payments.record</code> , <code>salesOrders.*</code> . Queries use plain <code>requireRole</code> / <code>requireUser</code> so the warning modal can still read.
<code>sessionFlags(ctx)</code>	<code>convex/lib/session.ts</code>	<code>users.me</code> (client gating for expired/2FA banners)
<code>heartbeat(ctx)</code>	<code>convex/lib/session.ts</code>	<code>convex/users.ts:heartbeat</code> (called every 60s from <code>useInactivity</code> on the client)
<code>audit(ctx, action, entity, entityId)</code>	<code>convex/lib/audit.ts</code>	Every successful mutation. Never throws on its own.

HELPER	FILE AND SIGNATURE	CALLED FROM
<code>enforce(ctx, class)</code>	<code>convex/lib/rateLimit.ts</code>	Every public mutation, after auth. 34 mutations across 12 files. Queries never call it.
<code>enforceDedup(ctx, fingerprint, windowMs=60000)</code>	<code>convex/lib/rateLimit.ts</code>	<code>convex/payments.ts:record</code> with <code>pay:\${invoiceId}:\${amount}:\${method}</code>
<code>windowStartFor(ts, windowMs)</code>	<code>convex/lib/rateLimit.ts</code>	Used inside <code>enforce</code>
<code>isRateLimitEnabled(ctx)</code>	<code>convex/lib/rateLimit.ts</code>	<code>enforce</code> , <code>enforceDedup</code>
<code>buildLineItemsForJob(ctx, jobId)</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>invoices.generate/regenerate/createEstimate/updateEstimate</code> , <code>jobs.syncInvoiceForJob</code>
<code>buildLineItemsForSalesOrder(ctx, salesOrderId)</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>invoices.generateSales/createEstimate/updateEstimate</code>
<code>nextInvoiceNumber(ctx, kind)</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>invoices.generate/generateSales/regenerate/createEstimate/convertEstimateToFinal</code>
<code>assertNotLocked(invoice)</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>invoices.generate/generateSales/regenerate/updateEstimate</code> , <code>jobs.addJobItem/removeJobItem</code> , <code>jobs.syncInvoiceForJob</code>
<code>findApprovedFinalForJob(ctx, jobId)</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>jobs.addJobItem/removeJobItem/reverseReady</code> , <code>invoices.convertEstimateToFinal</code> , <code>payments.record</code>
<code>findApprovedFinalForSalesOrder</code>	<code>convex/lib/invoiceHelpers.ts</code>	<code>invoices.convertEstimateToFinal</code> (sales branch)
<code>totpVerify / generateSecret / buildOtpauthUri</code> etc.	<code>convex/lib/totp.ts</code>	<code>convex/twoFactor.ts: setup/verifySetup/verifyLogin/disable</code> , <code>convex/users.ts flags</code>
<code>computeInvoiceTotals(lineItems, vatRate)</code>	<code>src/lib/schemas/invoice.ts</code>	<code>convex/lib/invoiceHelpers.ts:computeAndInsertTotals</code> , <code>prepareTotalsForJob</code> , client preview in <code>job.\$id.tsx</code>

convex/users.ts

SYMBOL	KIND	ARGS	GATE	RATE-LIMIT	AUDIT
<code>me</code>	query	<code>{}</code>	<code>getCurrentUser</code>		
<code>adminExists</code>	query	<code>{}</code>	<code>requireUser</code>		
<code>list</code>	query	<code>{}</code>	<code>requireRole(['admin','audit'])</code>		
<code>heartbeat</code>	mutation	<code>{}</code>	<code>requireUser</code> via <code>sessionHeartbeat</code>		
<code>setRole</code>	mutation	<code>{userId, role}</code>	<code>requireRole(['admin'])</code>	<code>enforce(admin)</code>	<code>audit</code>
<code>setActive</code>	mutation	<code>{userId, active}</code>	<code>requireRole(['admin'])</code>	<code>enforce(admin)</code>	<code>audit</code>
<code>bootstrapFirstAdmin</code>	mutation	<code>{}</code>	<code>getCurrentUser</code> + zero-admin check	<code>enforce(admin)</code>	<code>audit</code>

SYMBOL	KIND	ARGS	GATE	RATE-LIMIT	AUDIT
adminResetPassword	mutation	{userId, tempPassword}	requireRole(['admin'])	enforce(admin)	audit
changePassword	mutation	{currentPassword?, newPassword}	requireUser	enforce(admin)	audit
clearMustChangePassword	mutation	{}	requireUser	enforce(standard)	

convex/twoFactor.ts

SYMBOL	KIND	GATE	RATE-LIMIT	NOTES
status	query	getCurrentUser		Returns {enabled, hasSecret}
setup	mutation	requireUser	enforce(admin)	generateSecret + buildOtpauthUri + store draft secret
verifySetup	mutation	requireUser	enforce(standard)	On success: totpEnabled=true, backupCodes, lastTotpVerifiedTs=now, audit
verifyLogin	mutation	requireUser	enforce(standard)	Backup codes consumed on use; drives /auth/verify-2fa
disable	mutation	requireUser	enforce(admin)	Clears secret, enabled, backup codes; audit
regenerateBackupCodes	mutation	requireUser	enforce(admin)	Fresh codes; audit
adminReset	mutation	requireRole(['admin'])	enforce(admin)	Clears target's 2FA; audit

convex/customers.ts

SYMBOL	KIND	GATE	VALIDATORS
search	query	requireUser	empty query returns take(50), e withSearchIndex('name') + withSearchIndex('search_phone') merge and dedup
get	query	requireUser	
getWithVehicles	query	requireUser	throws if not found
create	mutation	requireActiveSession(['csr', 'salesRep', 'manager', 'admin'])	createCustomerSchema.parse by_phone duplicate guard (exact phone or Levenshtein ≤2 on name)
update	mutation	requireActiveSession(['csr', 'salesRep', 'manager', 'admin'])	updateCustomerSchema.parse phone collision guard if phone changed

Both mutations call `audit('customer.*')`.

convex/vehicles.ts

SYMBOL	KIND	GATE	VALIDATORS
get	query	requireUser	
byCustomer	query	requireUser	withIndex('ownerId')
byPlate	query	requireUser	normalizes trim().toUpperCase before lookup
inventory	query	requireUser	
create	mutation	requireActiveSession(['csr', 'salesRep', 'inventoryManager', 'manager', 'admin'])	createVehicle + plate toUpperCase regex ^[A-Z0-9]{2,}\$
adjustStock	mutation	same gate	non-negative quantity stockQty
update	mutation	same gate	updateVehicle + plate normalization present

All three mutations call `enforce(standard)` and `audit('vehicle.*')`.

convex/vehicleBrands.ts

SYMBOL	GATE	RATE-LIMIT	NOTES
list	requireUser		
create	requireActiveSession(['admin', 'manager'])	enforce(admin)	trims, normalizedName=lower+trim, uniqueness via <code>by_normalizedName</code>
update	same	same	uniqueness check excluding self
remove	same	enforce(admin)	

All three mutations call `audit('vehicleBrand.*')`.

convex/parts.ts

SYMBOL	KIND	GATE	RATE-LIMIT	VALIDATORS
list/get/lowStock/search/categories/brands/movements	query	requireUser		search takes {q?, brand?, category?} filter; movements is withIndex('partId') order desc take(10)
createPart	mutation	requireActiveSession(['inventoryManager', 'manager', 'admin'])	standard	createPartSchema.normalizeBrandCategory
updatePart	mutation	same	standard	updatePartSchema.empty brand/category

SYMBOL	KIND	GATE	RATE-LIMIT	VALIDATORS
<code>adjustStock</code>	mutation	same	<code>standard</code>	stock guard, writes <code>stockMovements +</code> <code>audit('stock.*')</code>
<code>importParts</code>	mutation	same	<code>bulk</code>	loops <code>createPartSchema.</code> per row, skips missing single <code>audit('parts.imo</code>

`convex/jobs.ts`

SYMBOL	KIND	GATE	RATE-LIMIT
<code>getDetail</code>	query	<code>requireUser</code>	
<code>byStatus</code>	query	<code>requireUser</code>	
<code>openCount</code>	query	<code>requireUser</code>	
<code>dashboardSummary</code>	query	<code>requireUser</code>	
<code>byCustomer</code>	query	<code>requireUser</code>	
<code>checkIn</code>	mutation	<code>requireActiveSession(['csr', 'manager', 'admin'])</code>	<code>standard</code>
<code>diagnose</code>	mutation	<code>requireActiveSession(['inventoryManager', 'manager', 'admin'])</code>	<code>standard</code>
<code>markReady</code>	mutation	same	<code>standard</code>
<code>complete</code>	mutation	<code>requireActiveSession(['manager', 'admin'])</code>	<code>standard</code>
<code>markPaid</code>	mutation	<code>requireActiveSession(['finance', 'manager', 'admin'])</code>	<code>financial</code>
<code>reverseReady</code>	mutation	<code>requireActiveSession(['manager', 'admin'])</code>	<code>standard</code>
<code>addJobItem</code>	mutation	<code>requireActiveSession(['inventoryManager', 'finance', 'csr', 'manager', 'admin'])</code>	<code>standard</code>
<code>removeJobItem</code>	mutation	<code>requireActiveSession(['finance', 'csr', 'manager', 'admin'])</code>	<code>standard</code>
<code>syncInvoiceForJob</code>	helper		

All job mutations call `audit('job.*')`.

`convex/appointments.ts`

SYMBOL	KIND	GATE	RATE-LIMIT
<code>list/listRange/upcoming/get</code>	query	<code>requireUser</code>	
<code>create</code>	mutation	<code>requireActiveSession(['csr', 'manager', 'admin'])</code>	<code>standard</code>
<code>markCheckedIn</code>	mutation	<code>same</code>	<code>standard</code>
<code>cancel</code>	mutation	<code>same</code>	<code>standard</code>

All three mutations call `audit('appointment.*')`.

`convex/invoices.ts` (the most gated surface)

SYMBOL	KIND	GATE	RATE-LIMIT	KEY GUARD
<code>getByJob</code>	query	<code>requireUser</code>		picks final ap first, strips g non-admin
<code>listByJob</code>	query	<code>requireUser</code>		strips generat
<code>listBySalesOrder</code>	query	<code>requireUser</code>		
<code>getById</code>	query	<code>requireUser</code>		admin-only ger
<code>generate</code>	mutation	<code>requireActiveSession(['finance', 'manager', 'admin'])</code>	<code>financial</code>	<code>assertNotLoc</code> paid guard, requ
<code>generateSales</code>	mutation	<code>requireRole(['finance', 'manager', 'admin', 'salesRep'])</code>	<code>financial</code>	same, via <code>buildLineIte</code>
<code>regenerate</code>	mutation	<code>requireActiveSession(['finance', 'manager', 'admin'])</code>	<code>financial</code>	<code>assertNotLoc</code>
<code>approve</code>	mutation	<code>requireActiveSession(['finance', 'manager', 'admin'])</code>	<code>financial</code>	rejects if already Sets <code>locked:t</code>
<code>createEstimate</code>	mutation	<code>requireRole(['csr', 'manager', 'admin', 'salesRep'])</code>	<code>financial</code>	<code>EST-YYYY-###</code> <code>status:'draf</code>
<code>updateEstimate</code>	mutation	<code>requireRole(['csr', 'manager', 'admin', 'salesRep'])</code>	<code>financial</code>	draft only, asse from source
<code>approveEstimate</code>	mutation	<code>requireRole(['finance', 'manager', 'admin'])</code>	<code>financial</code>	draft to approve lock.
<code>rejectEstimate</code>	mutation	<code>requireRole(['finance', 'manager', 'admin'])</code>	<code>financial</code>	reason 3..300,
<code>convertEstimateToFinal</code>	mutation	<code>requireRole(['finance', 'manager', 'admin'])</code>	<code>financial</code>	approved estim existing approve ####
<code>adminUnlock</code>	mutation	<code>requireRole(['admin'])</code>	<code>financial</code>	reason 10..300, <code>locked===tru</code> <code>locked:false</code>

convex/payments.ts

SYMBOL	KIND	GATE	RATE-LIMIT
byInvoice	query	requireUser	
record	mutation	requireActiveSession(['finance', 'manager', 'admin'])	financial + 60s dedup

record validates recordPaymentSchema.parse, rejects estimates, unapproved invoices, locked-and-paid invoices, and amounts exceeding grandTotal - amountPaid. Method enum: cash, transfer, card, pos, bank.

Other files, condensed

convex/leads.ts: list/search/get queries require user. create, updateStage, logFollowUp mutations require active session for csr, salesRep, manager, or admin. Rate-limited at standard.

convex/salesOrders.ts: get/list/byVehicle/byLead queries require user. create validates vehicle inStock + lead existence, auto-reserves the vehicle. complete, cancel, addPayment all require active session, rate-limited at financial.

convex/labourTypes.ts: list query requires user. create/update/remove require active session for manager or admin, rate-limited at financial.

convex/settings.ts: get query requires user. setVatRate requires active session for manager or admin, rate-limited at financial, Zod 0..100.

convex/deliveries.ts: get/getBySalesOrder queries require user. complete requires active session for salesRep, manager, or admin. Validates checklist booleans and salesOrder.status==='completed'. Rate-limited at financial.

convex/backfillPlates.ts: admin-only, requireActiveSession(['admin']), enforce admin, audit backfill.plates.

convex/rateLimit.ts: listEvents, getStatus, setEnabled require admin or audit. logEvent and cleanup are internal.

convex/activityLogs.ts: log mutation requires user. list query requires admin, manager, or audit.

convex/auditLogs.ts: list and distinctActions queries require admin or audit.

convex/migrations.ts: cleanupLegacyJobs, cleanupLegacyUsers, previewLegacyJobs, previewLegacyUsers. All use getAdminOrSystem which allows CLI calls (null identity) and admin UI, but rejects non-admin app users. Preview functions are read-only. Cleanup functions are idempotent and audited. See section 7 for migration verification.

convex/seed.ts and **convex/seedAdvanced.ts**: demo data mutations, not role-gated beyond being dev-only. They call audit and handle generatedById.

Validator matrix. Every public function declares args: { ... v.* }. Convex rejects shape and type errors before the handler runs. Inside, most mutations call Schema.parse from src/lib/schemas/*. moneyKobo is z.number().int().nonnegative(). vatRate is 0..100. Plates are ^[A-Z0-9]{2,}\$ after toUpperCase().

3. Encapsulation and inheritance

I want to be honest here. TypeScript and Convex do not have classical OOP class inheritance. There are no base classes, no extends, no virtual dispatch, no protected fields. The codebase gets encapsulation through module boundaries, closure-enforced invariants, and composition. That is the idiomatic way to do it on this stack, and pretending otherwise would mislead a reviewer.

What encapsulation means here

MECHANISM	HOW IT WORKS	EXAMPLE
Module boundary	<code>convex/</code> files are the trust boundary. Only <code>export const foo = query/mutation(...)</code> is callable from the client. Everything else is private to the module.	<code>convex/customers.ts:isVerySimilarName</code> and <code>levenshtein</code> are unexported pure functions used only inside <code>create</code> and <code>update</code> .
Default-deny auth wrapper	Every public function starts with a guard that throws. No mutation is callable without an explicit role allow-list. A null role is never authorized.	<code>convex/lib/auth.ts:requireRole</code> calls <code>src/lib/auth-utils.ts:isAuthorized</code> , which returns false for null or unknown roles. Admin bypasses. Audit role is explicit.
Composition over inheritance	<code>requireActiveSession</code> wraps <code>requireRole</code> . It adds inactivity and 2FA checks on top, without duplicating the role logic.	<code>requireActiveSession = requireRole + inactivity check + 2FA check</code> . Every <code>jobs.*</code> and <code>customers.*</code> write calls this one combinator.
Lib helpers hide invariants	Locking, window math, TOTP crypto, and line-item building live behind small pure APIs. Changing the lock rule touches one file.	<code>convex/lib/invoiceHelpers.ts:assertNotLocked</code> , <code>findApprovedFinalForJob</code> , <code>nextInvoiceNumber</code> (year rollover), <code>convex/lib/rateLimit.ts>windowStartFor</code> , <code>convex/lib/totp.ts:totpVerify</code>
Zod + v validators as contracts	<code>v.*</code> is the wire-level contract (Convex enforces before the handler). Zod is the domain contract (trim, regex, money bounds, enum, conditional fields). The handler never trusts raw args.	<code>convex/jobs.ts:addJobItem</code> declares <code>type: v.union('part','labour')</code> plus <code>addJobItemSchema.parse</code> which enforces "part needs partId, labour needs labourTypeId".
Component composition in React	No class components, no <code>extends React.Component</code> . UI reuses invariants via hooks and small components.	<code>useInactivity</code> drives <code>InactivityWarningModal</code> . <code>useCurrentUser</code> drives role-gated nav. <code>PrintableInvoice</code> and <code>PrintableJobCard</code> encapsulate print layout.

Why not classical inheritance

A textbook OOP design would introduce `abstract class Mutation { authorize(); validate(); execute(); }` with subclasses like `FinancialMutation extends Mutation`. In Convex that would be an anti-pattern for three reasons.

First, Convex handlers must be plain `mutation({args, handler})` values. The framework extracts args validators for codegen. A class hierarchy would hide validators from codegen and break `bunx convex codegen`.

Second, transactions are closures over `ctx`. The mutation handler IS the transaction scope. Subclass dispatch inside a transaction would obscure the atomic boundary.

Third, composition already gives the same reuse with fewer lines and explicit ordering. `await requireActiveSession(ctx, roles); await enforce(ctx, "financial"); const parsed = Schema.parse(args);` is three lines you can reorder with intention. A class would fix the order in one place that every subclass must remember to call `super.validate()` or it silently breaks. Composition keeps the security-critical call order visible at each call site, which reviewers can audit.

Concrete composition examples:

Auth composes. `requireActiveSession` composes `requireRole`, which composes `requireUser` + `isAuthorized`. TOTP verification in `twoFactor.ts:verifyLogin` composes `totpVerify` + backup code consumption without knowing about session inactivity. The session check composes again later in `requireActiveSession` for the next write.

Rate limiting composes into mutations. Every financial mutation reads `await requireRole; await enforce(ctx, "financial");`. The limit is not a subclass of "financial mutation". It is a one-line helper that reuses `getCurrentUser` and `windowStartFor`. The mutation keeps its own `v` + Zod validators. No shared base class needed.

Invoice line-item building composes. `buildLineItemsForJob` and `buildLineItemsForSalesOrder` are standalone `Promise.all` helpers that `invoices.*` and `jobs.syncInvoiceForJob` call. Same totals logic for service and sales, no `InvoiceBuilder` class hierarchy. `nextInvoiceNumber` with year rollover is reused by estimates and finals.

What is NOT here: no `extends` , no `super` , no mixins. Encapsulation here is narrow public API plus private helpers plus composed guards. That is the TypeScript/Convex equivalent of the encapsulation goal.

4. Table rules

All tables are defined in `convex/schema.ts:defineSchema` . The `authTables` spread comes first. Each `defineTable({...}).index(...).searchIndex(...)` line is the source of truth.

Table-by-table

TABLE	FIELDS (KEY SUBSET)	INDEXES / SEARCHINDEXES	WRITE RULE	IMMUTABILITY / L
users	name?, email?, role?, active?, totpSecret?, totpEnabled?, backupCodes?, lastTotpVerifiedTs?, lastActiveTs?, mustChangePassword?	index('email', ['email'])	setRole/setActive/adminResetPassword require admin. heartbeat requires user. changePassword requires user. bootstrapFirstAdmin requires zero admins.	Mutable. Audit cover changes.
customers	name, phone, email?, address?	index('by_phone') , searchIndex('name') , searchIndex('search_phone')	create / update require active session for csr, salesRep, manager, or admin	Mutable. Duplicate phone: exact match Levenshtein ≤2 on
vehicles	ownerId?, make, model, year, color, vin?, plate?, cost?, sellingPrice?, status, stockQty?, reorderLevel?	index('status') , index('owner') , index('by_plate') , searchIndex('search_plate')	create/update/adjustStock require active session for csr, salesRep, inventoryManager, manager, or admin	Mutable. Plate always toUpperCase()
vehicleBrands	name, normalizedName	index('by_normalizedName')	create/update/remove require active session for admin or manager	Mutable with case-uniqueness
jobs	vehicleId, customerId, csrId, status, complaint, diagnosis?, diagnosedById?, checkInTs, diagnosedTs?, inProgressTs?, readyForPickupTs?, completedTs?, paidTs?, reversedReadyTs?	index('status') , index('customerId')	checkIn for csr/manager/admin. diagnose/markReady for inventoryManager/manager/admin. complete/reverseReady for manager/admin. markPaid for finance/manager/admin. addJobItem/removeJobItem see inventory table.	Status machine via canTransition . reverseReady b approved final exist addJobItem/ removeJobItem b after completed/pa invoice locked.
jobItems	jobId, type, partId?, labourTypeId?, qty, unitPrice, lineTotal	index('jobId')	Only via jobs.addJobItem/ removeJobItem	removeJobItem r dispatched parts. A transitions job to inProgress .

TABLE	FIELDS (KEY SUBSET)	INDEXES / SEARCHINDEXES	WRITE RULE	IMMUTABILITY / L
invoices	jobId?, salesOrderId?, domain?, kind?, invoiceNumber?, status?, rejectedReason?, lineItems, partsTotal, labourTotal, subtotal, vat, grandTotal, approved, approvedTs?, paid, amountPaid, locked?, generatedById?	index('jobId') , index('salesOrderId') , index('invoiceNumber')	generate/regenerate/ approve require active session for finance/ manager/admin. generateSales adds salesRep. createEstimate/ updateEstimate add csr. approve/reject/convert for finance/ manager/admin. adminUnlock for admin only. All rate-limited at financial .	Lifecycle: draft esti (locked:false) approved (estimate lock) to converted plus new final draft to approved final se locked:true . Af assertNotLocke findApprovedFin blocks writes. Only adminUnlock cle locked . paid w amountPaid >= grandTotal . Paid cannot be regenera
labourTypes	name, fixedPrice	none	create/update/remove require active session for manager or admin	Mutable
parts	code, description, costPrice, sellingPrice, stockQty, reorderLevel, brand?, category?	index('by_code') , index('by_brand') , index('by_category') , searchIndex('search_code')	createPart/ updatePart/ adjustStock/ importParts require active session for inventoryManager, manager, or admin	Mutable. updateP aliases partNumbe code .
stockMovements	partId, qty, type, jobId?, ts, userId	index('partId')	Only via parts.adjustStock and jobs.addJobItem (part path)	Append-only. No delete mutations ex
payments	invoiceId, amount, method, ts, recordedById	index('invoiceId')	payments.record requires active session for finance/manager/admin + 60s dedup	Append-only. No delete. Guards: fina approved only, am remaining balance, enum.
leads	name, phone, email?, interestedVehicleId?, stage, notes[], nextFollowUpTs?	index('stage') , searchIndex('name') , searchIndex('phone')	create/updateStage/ logFollowUp require active session for csr, salesRep, manager, or admin	Mutable
salesOrders	vehicleId, leadId, agreedPrice, deposit, balance, reservedTs, status, payments?	index('vehicleId') , index('leadId')	create/complete/ cancel/addPayment require active session for csr, salesRep, manager, or admin	Mutable. create reserves vehicle.
deliveries	salesOrderId, checklist, handedOverTs, repId	index('salesOrderId')	complete requires active session for salesRep, manager, or admin	Insert-only (one de order)

TABLE	FIELDS (KEY SUBSET)	INDEXES / SEARCHINDEXES	WRITE RULE	IMMUTABILITY / L
appointments	customerId?, name?, phone?, email?, vehicleMake?, vehicleModel?, vehiclePlate?, complaint?, appointmentTs, status, createdById, checkInJobId?	index('appointmentTs') , index('status') , index('phone') , index('customerId')	create/markCheckedIn/ cancel require active session for csr, manager, or admin	Status machine: sc checkedIn or cancel rows require cust (t1).
auditLogs	userId, action, entity, entityId, ts	index('entityId') , index('by_user') , index('by_ts')	No direct write mutations. Only convex/lib/ audit.ts:audit inside other mutations.	Append-only, imm No update or delete audit() silently no actor.
activityLogs	userId?, email?, event, ts, userAgent?, browser?, device?, screenInfo?, ip?	index('by_user') , index('by_ts') , index('by_event')	log mutation requires user. list query requires admin, manager, or audit.	Append-only. Serv null in pure mutatio limitation: only HTT could capture IP).
settings	vatRate, nextEstSeq?, nextInvSeq?, estYear?, invYear?, rateLimitEnabled?	none	setVatRate requires active session for manager or admin. rateLimit.setEnabled requires admin.	Singleton (first sett nextInvoiceNum mutates seq/year a
rateLimits	key, windowStart, count, actionClass	index('by_key_window')	Only via lib/ rateLimit.ts:enforce/ enforceDedup and GC cleanup	One doc per user:class:wind Pruned after 24h.
rateLimitEvents	key, actionClass, ts, limit, windowMs, retryAfterMs, userId?	index('by_ts') , index('by_actionClass')	enforce best-effort insert before throw. logEvent internal. cleanup prunes >30d.	Observability, best- Rolls back on throw mutation.
authTables	authSessions, authAccounts, authVerificationCodes, ...		Managed by Convex Auth	

Relation integrity. Convex has no declarative foreign keys with cascade. FK discipline is enforced in code. `jobs.checkIn` checks `vehicleId` and `customerId` existence. `salesOrders.create` checks `vehicleId` and `leadId`. `appointments.create` checks `customerId`. `jobs.addJobItem` checks `partId` and `labourTypeId`. `payments.record` checks `invoiceId`. No delete mutations are exposed, so dangling refs are not created after the demo.

5. How Convex works

This section is for a reviewer who may not know Convex.

What Convex is. Convex is a reactive backend-as-a-service. You define documents (JSON-like objects) in tables, write server functions, and the client subscribes to query results that push updates automatically. There is no hand-written REST layer, no ORM, no separate database server to tune. The deployment runs your `convex/` functions and stores documents in Convex's managed document store.

Documents. A document is a JSON object with a system `_id: Id<Table>` and `_creationTime: number`. Fields are typed by `v.*` validators. Tables are created by `defineTable({...})` in `convex/schema.ts`. Adding a table or field is additive. Old documents stay readable because new fields use `v.optional`. The `authTables` spread contributes `users`, `authSessions`, `authAccounts`, and friends for Convex Auth.

Transactions. Every `query` or `mutation` handler runs in a single ACID transaction. All `ctx.db` calls inside the handler are atomic. Either all writes commit or none do. If a throw happens after a write, the whole transaction rolls back. Concurrency uses optimistic concurrency control (OCC). Convex does not lock rows on read. If two mutations concurrently patch the same document, one commits and the other retries or fails with a conflict. This matters for rate limiting. A single global counter doc per class would become a hot-row bottleneck, with every finance writer contending on one doc. The shipped design avoids that by sharding per `user:class>windowStart`, so writers contend only with themselves in the same minute.

Queries vs mutations vs actions.

KIND	READS	WRITES	TRANSACTIONS	USE IN THIS CODEBASE
<code>query</code>	yes	no	read-only, cached, reactive	All <code>list/search/get/by*</code> reads
<code>mutation</code>	yes	yes	single ACID transaction	Every <code>create/update/generate/approve/record</code>
<code>action</code>	yes (via <code>ctx.runQuery/runMutation</code>)	via mutations only	not transactional across the action	<code>convex/seed.ts:seed</code> (seeds demo data)
<code>internalMutation</code>	yes	yes	transaction, not callable from client	<code>convex/rateLimit.ts:logEvent/cleanup</code> (called by cron)
<code>httpAction</code> / <code>httpRouter</code>				<code>convex/http.ts</code> exposes <code>auth.addHttpRoutes(http)</code> so <code>POST /api/auth/signIn</code> works

Reactive subscriptions. A `useQuery(api.jobs.getDetail, {jobId})` is not polling. The client opens a subscription. When any document read by that query changes, Convex pushes a fresh result to all subscribers. This is why the dashboard and job detail stay consistent without manual refetch. `queryClient.invalidateQueries()` is a secondary belt.

Scheduler and crons. `convex/crons.ts` registers `crons.daily("rateLimit cleanup", {hourUTC:3}, internal.rateLimit.cleanup, {})`. At 03:00 UTC daily, the cleanup internal mutation scans `rateLimitEvents` and `rateLimits`, deletes rows older than 30 days and 24 hours respectively. No per-mutation scheduler is used on the hot path.

Auth. `convex/auth.ts` configures `convexAuth({providers:[Password]})` from `@convex-dev/auth`. User identity inside any handler is `getAuthUserId(ctx)`. The `users` table holds profile fields plus the t3 security extensions (`totp*`, `lastActiveTs`, `mustChangePassword`). Password hashes live in `authAccounts.secret` (Script via `modifyAccountCredentials`), never in `users`. The `sendVerificationRequest` in `auth.ts` was silenced to fix CR-01. It no longer logs `identifier`, `token`, or `url`.

Client mapping. `src/lib/queries.ts` wraps `api.*` with TanStack Query keys so React components call `jobQueries.detail(jobId)` and similar. `src/routes/*` file routes render those queries and mutations. Validation types are generated by `bunx convex codegen` into `convex/_generated/dataModel.ts` and `api.ts`.

6. Validators and locks

How `v.object` runs on every public function

Convex `v` runs before the handler. Each `query` or `mutation` declares `args: { field: v.xxx }`. Convex validates the client-supplied args against that shape before the handler body starts. A mismatch returns an error to the client without entering the transaction.

Examples:

- `convex/jobs.ts:addJobItem` declares `type: v.union(v.literal('part'), v.literal('labour'))`, `partId: v.optional(v.id('parts'))`, `labourTypeId: v.optional(v.id('labourTypes'))`, `qty: v.number()`, `unitPrice: v.number()`.
- `convex/payments.ts:record` declares `method: v.union(v.literal('cash'), v.literal('transfer'), v.literal('card'), v.literal('pos'), v.literal('bank'))`. The allow-list is enforced at the wire boundary.
- `convex/parts.ts:adjustStock` derives `type: v.union(...STOCK_MOVEMENT_TYPES.map(t=>v.literal(t)))` from the single `src/lib/enums.ts` source, so `v` and Zod stay aligned.
- `convex/invoices.ts:adminUnlock` declares `reason: v.string()`. The length rule (10 to 300 chars after trim) is a second-layer business guard.

Zod runs inside the handler, after auth. Most mutations call `Schema.parse(args)` from `src/lib/schemas/*`. This is the domain-level contract: trims, regexes, bounds that `v` alone cannot express. `v` knows `v.string()`. Zod knows `z.string().trim().min(1).max(128)` plus `plateValidator (/^[A-Z0-9][A-Z0-9-]{2,}$/)`. The handler throws a `ZodError` (surfaced as `ConvexError`) if Zod fails, still before any `ctx.db` write. The pattern is always: `require*` / `enforce` / `Schema.parse` / business guard / `ctx.db.*` / `audit`.

Invoice lock

Data field. `invoices.locked?: boolean` in `convex/schema.ts`. Absent means false. Old finals before t5 remain mutable until approved.

Trigger 1: locking at approve. `convex/invoices.ts:approve` is the only handler that sets `locked:true`. Not `approveEstimate`. Just `approve`.

```
// convex/invoices.ts:285 approve
await ctx.db.patch(args.invoiceId, {
  approved: true,
  approvedTs: Date.now(),
  status: 'approved',
  locked: true, // ← trigger
  generatedById: (invoice as any).generatedById ?? user._id,
})
```

Before the patch: the handler checks the invoice exists, is not already approved, and is not an estimate (estimates use `approveEstimate`). After this patch, `locked === true`.

No other handler sets `locked:true`. `generate`, `regenerate`, `generateSales`, `createEstimate`, and `convertEstimateToFinal` all insert with `locked:false`. Drafts are intentionally unlocked so job items can still sync. `approveEstimate` patches `status:'approved'` and `approved:true` but does NOT set `locked`. Estimates must stay editable until converted. Only finals lock.

Trigger 2: where `locked` is read as a guard. This is the mechanism that actually blocks writes after approval.

GUARD SITE	FILE:SYMBOL	CONDITION THAT BLOCKS	ERROR THROWN
<code>invoices.generate</code>	<code>convex/invoices.ts:generate</code>	<code>existingFinal</code> and <code>assertNotLocked(existingFinal)</code> or paid	"Invoice is locked, unlock by Admin required." / "Cannot regenerate an invoice that is already paid."
<code>invoices.generateSales</code>	<code>convex/invoices.ts:generateSales</code>	<code>assertNotLocked</code>	same

GUARD SITE	FILE:SYMBOL	CONDITION THAT BLOCKS	ERROR THROWN
<code>invoices.regenerate</code>	<code>convex/invoices.ts:regenerate</code>	<code>assertNotLocked(existing)</code>	same
<code>invoices.updateEstimate</code>	<code>convex/invoices.ts:updateEstimate</code>	<code>assertNotLocked(invoice)</code>	same
<code>jobs.addJobItem</code>	<code>convex/jobs.ts:addJobItem</code>	<code>findApprovedFinalForJob</code> non-null OR any invoice with <code>locked===true</code>	"Cannot add items: final invoice is locked."
<code>jobs.removeJobItem</code>	<code>convex/jobs.ts:removeJobItem</code>	same, plus <code>type==='part'</code> always rejected	same
<code>jobs.syncInvoiceForJob</code>	<code>convex/jobs.ts:syncInvoiceForJob</code>	if <code>locked</code> , sync silently no-ops so the ledger does not drift	
<code>payments.record</code>	<code>convex/payments.ts:record</code>	<code>locked && paid</code> rejected, plus balance cap	"Invoice is locked and already paid." / "Payment amount exceeds remaining balance."
<code>jobs.reverseReady</code>	<code>convex/jobs.ts:reverseReady</code>	<code>findApprovedFinalForJob</code> non-null	"Cannot reverse ready-for-pickup after final invoice is approved/locked."
<code>invoices.convertEstimateToFinal</code>	<code>convex/invoices.ts:convertEstimateToFinal</code>	checks no existing approved final for same job/order	"An approved final invoice already exists for this job/order."

The helper `findApprovedFinalForJob` simply returns `kind==='final' && approved`. An approved final is treated as locked even if a caller forgot to check `locked`. The code double-guards: `assertNotLocked(existingFinal)` plus `allInvoices.some(i=>i.locked)`.

Unlock path. `convex/invoices.ts:adminUnlock` is the only mutation that patches `locked:false`. It requires `requireRole(['admin'])`, a reason between 10 and 300 characters, and only works if the invoice is actually locked. The reason is audited (first 80 chars). After unlock, the final becomes `approved:true, locked:false`, so `generate / regenerate / addJobItem` would work again until the next `approve` re-locks.

generatedById tracking. On first `generate` and on `approve`, the final records which user produced or approved it. Queries project `generatedById` only for `role==='admin'`. Non-admin users receive the invoice without that field. This is the admin-only audit field, not a lock, but it is part of the locking PRD. Provenance survives after `locked`.

Rate-limit triggers

Every public mutation throws `RATE_LIMITED` when its per-user per-class budget is exhausted. The class is wired in each mutation (see section 2). In code, this is one line inserted after `require*` and before `Schema.parse`, so it never leaks quota to unauthenticated calls:

```
// pattern repeated in 34 mutations
const user = await requireActiveSession(ctx, [...])
await enforce(ctx, "financial")           // or "admin" | "bulk" | "standard"
const parsed = SomeSchema.parse(args)    // Zod
```

Classes:

- **admin 5/min:** `users.*`, `vehicleBrands.*`, `backfillPlates`, `twoFactor.setup/disable/regenerate/adminReset`
- **financial 20/min:** `payments.record + dedup`, `invoices.*` (10 functions), `salesOrders.*` (4), `jobs.markPaid`, `settings.setVatRate`, `labourTypes.*`, `deliveries.complete`
- **bulk 5/min:** `parts.importParts`
- **standard 60/min:** everything else (customers, vehicles, leads, appointments, most job mutations, parts CRUD)

Queries never call `enforce`. They are read-only and rate-limiting them would hurt a busy shift. The dedup helper `enforceDedup(ctx, 'pay:invoice:amount:method')` fires additionally in `payments.record`. A second identical payment within 60 seconds throws `DEDUP` even if the 20/min bucket would allow it. This prevents double-click ledger noise.

Window math. `windowStartFor(now, 60000) = floor(now/60000)*60000`. The first mutation in a window inserts `rateLimits:{key:userId:class, windowStart:ws, count:1}`. Later mutations in the same window patch `count+1`. At `count >= limit` the throw includes `retryAfterMs = ws+windowMs - now` so the client can show a countdown. The next wall-minute gets a fresh `ws` and a fresh doc, so there is no contested reset. The cron at 03:00 UTC prunes `windowStart < now-24h` and `rateLimitEvents.ts < now-30d`.

Session-expiry rejection path

`requireActiveSession` throws `ConvexError("Session expired due to inactivity. Please sign in again.")` when `now - lastActiveTs > 30*60*1000`. On the client, `useInactivity` debounces user events, heartbeats `api.users.heartbeat` every 60 seconds (server throttles to 50s), shows `InactivityWarningModal` at 25 minutes with an "Extend" button, and hard-redirects to `/auth/login?expired=1` at 30 minutes. `users.me` returns `sessionFlags` so the router can gate `/auth/verify-2fa` and `/auth/change-password`. Until `lastActiveTs` is set (fresh login before first heartbeat), the server allows the first write. Once set, every `requireActiveSession` write carries the same 30-minute clock. 2FA adds its own `lastTotpVerifiedTs` gate with the same window.

7. Findings from v1, and where they stand now

CMA-CRR-001 (07 Aug 2026) findings, reconciled at the current HEAD:

ID	V1 OBSERVATION	STATUS
CR-01	<code>convex/auth.ts:sendVerificationRequest</code> logged token and URL to console	Fixed in t3. The function now returns immediately with a comment: "Never log tokens or URLs".
CR-02	<code>payments.record</code> could exceed <code>grandTotal</code>	Fixed. Balance cap throws if <code>amount > grandTotal - amountPaid</code> . Method enum enforced. <code>salesOrders.addPayment</code> already capped.
CR-03	No rate limiting	Fixed in t7. <code>convex/lib/rateLimit.ts</code> + 34 mutations wired, ships enabled, admin kill-switch. Auth HTTP is not rate-limited inside mutations (no IP in <code>ctx</code>). Client uses <code>isPending</code> debounce instead. Documented honestly.

ID	V1 OBSERVATION	STATUS
CR-04	<code>invoices.generate</code> vs <code>regenerate</code> duplication	Fixed. <code>buildLineItemsForJob</code> , <code>buildLineItemsForSalesOrder</code> , <code>prepareTotalsForJob</code> , and <code>nextInvoiceNumber</code> unify the duplication. <code>jobs.syncInvoiceForJob</code> is consistent.
CR-05	<code>parts.adjustStock</code> used <code>throw new Error</code> instead of <code>ConvexError</code>	Fixed. Now uses <code>ConvexError</code> .
CR-06	<code>parts.search</code> collected all rows then filtered in JS	Kept. <code>search</code> AND-combines <code>q</code> , <code>brand</code> , <code>category</code> but still collects then filters. At seed scale (45 parts) this is fine. A bounded <code>take()</code> may come later if scale grows.
CR-07 to CR-13	Route-guard duplication, <code>bootstrapFirstAdmin</code> race, <code>audit()</code> silent skip, <code>auditLogs</code> index, <code>v.string()</code> frontiers, <code>window.innerWidth</code> hydration, <code>importParts</code> loop inserts	Documented backlog (Low/Perf). <code>importParts</code> still loop-inserts because Convex has no bulk insert. <code>auditLogs</code> has <code>by_user</code> and <code>by_ts</code> indexes.

Additional work since v1:

- **t1:** plate normalization, customer dedup (phone exact match + Levenshtein ≤ 2 on name)
- **t2:** brand/category, `vehicleBrands` with `by_normalizedName` uniqueness
- **t3:** TOTP 2FA, session timeout, force-password-change
- **t4:** `audit` role, `activityLogs`
- **t5:** sales/service split (`domain` , `kind` , `invoiceNumber` , `locked` , `generatedById`), `reverseReady` guard, sold-vehicle stock guard, invoice locking
- **t7:** throttling (34 mutations, 4 classes, dedup, cron cleanup)
- **Legacy migration:** `convex/migrations.ts` added to remove legacy `technician` role and job fields (`technicianId` , `assignedTs` , `waitingReleaseTs`) and normalize legacy statuses (`assigned` to `checkedIn` , `waitingRelease` to `diagnosed`). Verified on prod: `previewLegacyJobs` returns `legacyCount: 0` across 12 jobs. `previewLegacyUsers` returns `legacyCount: 0` across 8 users. Both cleanup functions are idempotent and audited.

8. Risks v1 and v2 skipped

The earlier reports cataloged what shipped and how it works. They did not flag the things below. Each is a real finding you can grep. None are blocking, but a reviewer should know about them.

Plaintext TOTP secrets in the users table

`convex/lib/totp.ts` is upfront about this in its header comment. TOTP secrets sit in `users.totpSecret` as plain strings. Convex encrypts data at rest, but there is no per-field envelope encryption in this app. Anyone with direct read access to the database can read every secret and generate valid codes. The same applies to backup codes (`users.backupCodes`). The codebase never logs secrets and audit entries record only `userId + action` , so the leak path is DB read access, not logs. If the threat model includes an insider with DB access, this matters. If it does not, the current design is acceptable and matches the PRD scope cut.

Silent `catch {}` blocks

12 `catch {}` blocks across `convex/` swallow errors with no logging:

- `convex/twoFactor.ts:137-138` and `175-176` patch `totpSecret` and `backupCodes` to `undefined` , then swallow the result. The intent is to clear optional fields that Convex sometimes refuses to patch to `undefined` . The pattern works but hides real write failures.
- `convex/users.ts:220` does the same for `mustChangePassword` .
- `convex/parts.ts:193` and `196` clear `brand` and `category` the same way.

- `convex/lib/rateLimit.ts:23` swallows settings read failures, defaulting to enabled.
- `convex/lib/rateLimit.ts:85` swallows the best-effort `rateLimitEvents` insert on throw.

If any of these silently fail, the app continues as if it worked. A reviewer should know that "clear field X" is not guaranteed to clear. The fix is low effort: log the swallowed error to `auditLogs` or an internal table before ignoring it.

adminResetPassword SHA-256 fallback

`convex/users.ts:adminResetPassword` tries to reset the password through `@convex-dev/auth/server` `modifyAccountCredentials`, which hashes with Scrypt. If that throws, the fallback path at `convex/users.ts:157-166` patches `authAccounts.secret` directly with a SHA-256 hex hash. That hash will never verify against Scrypt, so the user cannot log in with the temp password. The code sets `mustChangePassword: true` and comments that the admin should use email reset instead. This is a known degraded path, not a bug. A reviewer should confirm the primary path works in their environment and treat the fallback as a last resort, not a normal flow.

Unbounded `collect()` reads

Several queries call `collect()` on whole tables with no `take()` bound:

- `convex/jobs.ts:100` and `113` collect all jobs for `dashboardSummary` and `openCount`.
- `convex/migrations.ts:46` collects all jobs for cleanup.
- `convex/jobs.ts:65` collects all jobs when no status filter is passed.

The Convex guidelines in `convex/_generated/ai/guidelines.md:327` explicitly warn against this: "instead of using `.collect()` use `.take()` or `paginate`". At demo scale (12 jobs, 8 users, 45 parts) this is fast. It will not stay fast if the tables grow. The queries should be bounded with `take()` or replaced with a denormalized counter for `openCount`. The earlier reports noted CR-06 (parts search collect-then-filter) as "kept, fine at seed scale". The same caveat applies here but was not written down.

as any casts

115 as any casts in `convex/`, concentrated in:

- `convex/invoices.ts` (30)
- `convex/twoFactor.ts` (26)
- `convex/users.ts` (13)
- `convex/lib/session.ts` (9)

Most cast optional schema fields (`totpSecret`, `backupCodes`, `mustChangePassword`, `locked`) that were added in t3/t5 but are still typed `v.optional(v.*)`. The casts exist because the Convex codegen types a document field as `T | undefined`, and the handler wants to write `undefined` to clear it. This is a known Convex friction point. The casts are not hiding bugs, but they defeat the type system at the exact points where security fields live. A future cleanup would use a typed helper or wait for Convex to support `patch(doc, { field: undefined })` without a cast.

No HTTP security headers

There is no `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security`, or `Referrer-Policy` set anywhere in `src/` or the Vite/Nitro config. TanStack Start with Nitro does not set these by default. For an internal app this is low risk. For a public-facing app, add at least a CSP and HSTS. The earlier reports did not mention headers at all.

Auth HTTP not rate-limited

CR-03 noted this and the report is honest about it. The sign-in HTTP endpoint (`POST /api/auth/signIn`) is not rate-limited because mutations have no `ctx.request IP`. The client uses `isPending` debounce. The earlier reports document this as a limitation. The concrete risk is credential stuffing against the password endpoint. A future fix would put a reverse proxy or Convex action in front that can see the IP and bucket by IP + identifier.

Test coverage is thin

The repo has 10 test files under `tests/`:

- `tests/unit/schemas.test.ts`, `job-utils.test.ts`, `auth-utils.test.ts`, `format.test.ts`, `invoice.test.ts`
- `tests/convex/authz.test.ts`, `phase2-authz.test.ts`
- `tests/e2e/jobs.spec.ts`, `screenshots.spec.ts`, `auth-nav.spec.ts`

The unit tests cover pure helpers (Zod schemas, job status transitions, auth role checks, money formatting, invoice totals). The Convex tests cover authorization. There are no tests for the invoice lock path, rate limiting, 2FA flows, or the migrations. The earlier reports cited "35 pass, 10 skip" from t8 but never listed what is actually tested. A reviewer should know the lock trigger in section 6 is proven by code reading, not by a test.

9. Verification

This report is read-and-document only. No `convex/*.ts`, `src/*`, or `convex/schema.ts` behavior changed to produce it.

Git diff

```
git diff main...HEAD --stat
```

Expected: only `docs/code-review-report-v3.md` and this PDF. No `convex/`, `src/`, `package.json`, `convex.json`, or migration files. If any `convex/*.ts` or `src/*` line appears, the commit is invalid.

File:symbol anchors are real

Every reference in this report is a real file and symbol. You can verify:

```
grep -n "export const generate\|export const approve\|export const adminUnlock" convex/invoices.ts
grep -n "export async function requireActiveSession\|export async function requireRole" convex/lib/session.ts
convex/lib/auth.ts
grep -n "export async function enforce\|export async function enforceDedup" convex/lib/rateLimit.ts
grep -n "export async function buildLineItemsForJob\|export function assertNotLocked" convex/lib/invoiceHelpers.ts
grep -n "locked" convex/schema.ts
grep -n "export const cleanupLegacyJobs\|export const previewLegacyJobs" convex/migrations.ts
```

Each returns the line number cited above.

Migration verification on prod

The legacy schema migration was verified against the production deployment (`watchful-reindeer-590`):

```
npx convex run migrations:previewLegacyJobs
# { "legacyCount": 0, "total": 12, "statuses": { "checkedIn": 3, "diagnosed": 3, "inProgress": 1,
"readyForPickup": 3, "completed": 1, "paid": 1 } }

npx convex run migrations:previewLegacyUsers
# { "legacyCount": 0, "total": 8 }
```

No `assigned` or `waitingRelease` statuses remain. No `technicianId`, `assignedTs`, or `waitingReleaseTs` fields remain. No `technician` role remains. Both preview functions are read-only and safe to re-run from the Convex dashboard under Functions, or from the CLI.

Build and typecheck

No app code changed, so typecheck and build remain green. The prior t8 verification recorded: `bun x tsc --noEmit` exit 0, `bun x vitest run --exclude="tests/e2e/**"` 35 pass / 10 skip, `bun run build` green. To re-run:

```
bun install          # once
bun x tsc --noEmit   # exit 0
bun x vitest run --exclude="tests/e2e/**" # 35 pass, 10 skip
bun run build        # green
```

Manual UI spot-check (optional)

Login creds are in `docs/mock-accounts.md` (password `password123` for all roles). Admin: `cedric@cedricmastersautos.com`.

1. Open any job at `/service/job/$id`. Generate a final invoice, approve it. The `LOCKED` banner appears. Try adding a part. It fails with "final invoice is locked."
2. Sign in as `audit@cedricmastersautos.com`. That account is read-only. No write buttons appear anywhere.
3. Navigate `/service/parts`. Brand and category filters work. Stock badge shows `LOW` for low-stock items. `Import CSV` button is present (rate-limited at `bulk`).

This report was generated 26 Aug 2026. Document ID CMA-CRR-003. Prior reports (`docs/code-review-report.html` , `docs/code-review-report.pdf` , `docs/code-review-report-v2.md`) are preserved and untouched.